Actividad Autónoma 09

Diferencias por plataforma y evidencia Android

Estudiante	Roberto Valencia
Asignatura	Desarrollo de Aplicaciones Móviles
Unidad y sesión	Unidad 2 · Sesión 3 (sesión 9 del curso)
Fecha	12 de octubre de 2026
Rama	feature/expect-actual-valencia
Repositorio	https://github.com/Akatosh4019/PharmaMobile_Valencia/tree/feature/expect-actual-valencia

Alcance de la evidencia

La aplicación y los tests se ejecutaron en Android. El código iOS está escrito, pero no se compiló ni ejecutó en Windows. El estudiante indicó que no se exige usar iOS por falta de Mac. No se presentan capturas ni resultados de iOS inventados.

Este PDF contiene los cuatro productos: inventario, informe comparativo, tercera capacidad y evidencias de ejecución. Los commits están en la rama enlazada.

Producto 1 · Inventario de capacidades nativas

Moneda: expect fun formatearSoles(valor: Double): String

commonMain: shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobile/platform/Formato.kt

Android: shared/src/androidMain/kotlin/pe/edu/upeu/pharmamobile/platform/Formato.android.kt; java.text.NumberFormat, Locale.forLanguageTag("es-PE")

iOS: shared/src/iosMain/kotlin/pe/edu/upeu/pharmamobile/platform/Formato.ios.kt; Foundation NSNumberFormatter, NSLocale("es_PE")

Información del dispositivo: expect class InfoDispositivo() con sistema y version

commonMain: shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobile/platform/InfoDispositivo.kt

Android: shared/src/androidMain/kotlin/pe/edu/upeu/pharmamobile/platform/InfoDispositivo.android.kt; Build.VERSION.RELEASE

iOS: shared/src/iosMain/kotlin/pe/edu/upeu/pharmamobile/platform/InfoDispositivo.ios.kt; UIDevice.currentDevice.systemName/systemVersion

Módulo DI: expect val platformModule: Module

commonMain: shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobile/di/AppModule.kt

Android: shared/src/androidMain/kotlin/pe/edu/upeu/pharmamobile/di/PlatformModule.android.kt; Koin module, motor Ktor OkHttp, CompartidorAndroid(get()); androidContext se proporciona en androidApp/src/main/kotlin/pe/edu/upeu/pharmamobile/MainApplication.kt

iOS: shared/src/iosMain/kotlin/pe/edu/upeu/pharmamobile/di/PlatformModule.ios.kt; Koin module, motor Ktor Darwin, CompartidorIos() sin Context

Identificación de plataforma: expect fun getPlatform(): Platform

commonMain: shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobile/Platform.kt

Android: shared/src/androidMain/kotlin/pe/edu/upeu/pharmamobile/Platform.android.kt; Build.VERSION.SDK_INT

iOS: shared/src/iosMain/kotlin/pe/edu/upeu/pharmamobile/Platform.ios.kt; UIDevice

Registro HTTP: expect fun loggerHttp(): Logger

commonMain: shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobile/data/remote/HttpClientFactory.kt

Android: shared/src/androidMain/kotlin/pe/edu/upeu/pharmamobile/data/remote/HttpLogger.android.kt; android.util.Log.i

iOS: shared/src/iosMain/kotlin/pe/edu/upeu/pharmamobile/data/remote/HttpLogger.ios.kt; Logger.SIMPLE

Compartidor: interfaz común, no expect/actual

Contrato: shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobile/domain/platform/Compartidor.kt. Android: shared/src/androidMain/kotlin/pe/edu/upeu/pharmamobile/platform/CompartidorAndroid.kt, con Intent.ACTION_SEND. iOS: shared/src/iosMain/kotlin/pe/edu/upeu/pharmamobile/platform/CompartidorIos.kt, con UIActivityViewController. Koin registra cada implementación en PlatformModule del source set correspondiente.

Producto 2 · Informe comparativo

Proyecto: PharmaMobil Valencia. Sesión: 9, Unidad 2, sesión 3. Plataformas: Android verificado; código iOS escrito, sin ejecución por falta de Mac. Este informe compara el código real del repositorio. Las observaciones exclusivas de iOS se identifican como análisis de código, no como resultado de simulador.

1. ¿Por qué cambia el formato de moneda aunque el locale sea peruano?

En platform/Formato.kt, el código común declara expect fun formatearSoles(valor: Double): String: pide un resultado, pero no decide cómo construirlo. Formato.android.kt usa NumberFormat.getCurrencyInstance(Locale.forLanguageTag("es-PE")), que pertenece a la biblioteca Java disponible en Android. Formato.ios.kt usa NSNumberFormatter y NSLocale("es_PE"), disponibles en Foundation. Ambas bibliotecas interpretan convenciones peruanas, pero tienen reglas de implementación, datos locales y símbolos que pueden diferir. En Android observamos el precio S/ 25.01 en el listado; la interfaz también mostró dos decimales y separador de millares para otros productos. No asignamos un resultado literal a iOS: Windows no permite compilar ni abrir el simulador. Aunque el idioma y país coincidan, asumir que las cadenas son idénticas sería una conclusión sin prueba. La comparación literal de espacios o separadores quedaría pendiente de una ejecución real en macOS.

2. ¿Por qué el compartidor Android necesita Context y el de iOS no?

CompartidorAndroid.kt construye un Intent.ACTION_SEND, agrega el texto mediante Intent.EXTRA_TEXT, crea un selector con Intent.createChooser y lo abre con contexto.startActivity. En Android, iniciar otra actividad requiere un Context. Como Koin recibe androidContext(this@MainApplication) en androidApp/.../MainApplication.kt, el módulo de androidMain puede crear CompartidorAndroid(get()). Además, el selector recibe FLAG_ACTIVITY_NEW_TASK porque se parte del contexto de aplicación. En cambio, CompartidorIos.kt crea un UIActivityViewController y lo presenta desde el controlador raíz de UIApplication.sharedApplication.keyWindow; no usa Context de Android. Por eso el módulo iOS construye CompartidorIos() sin argumento. Ambos cumplen la misma interfaz Compartidor, pero sus mecanismos de navegación y las dependencias que Koin entrega son distintos. El selector Android se verificó en el emulador; la hoja iOS se deduce del código, no se ha visto funcionando.

3. ¿Por qué combinar expect/actual e interfaz con inyección?

Podríamos haber definido una interfaz FormateadorMoneda y registrado dos implementaciones en Koin. Eso facilitaría reemplazarla por un falso en pruebas y haría explícita la dependencia de la pantalla, pero añadiría un tipo y un registro de inyección para una operación pequeña y pura. Elegimos expect/actual para formatearSoles: la llamada desde ProductoViewModel.kt queda directa y el compilador exige una implementación por plataforma. La desventaja es que una prueba común no puede sustituir la función tan sencillamente. Para compartir elegimos la solución opuesta: Compartidor es una interfaz de dominio, porque abrir el selector produce un efecto externo y depende del ciclo de vida de la interfaz. Koin permite inyectar un sustituto en pruebas sin abrir una aplicación externa. Esta diferencia no es arbitraria: usamos expect/actual cuando la capacidad es pequeña y su elección es de compilación, e interfaz cuando importa controlar dependencias y efectos.

4. ¿Qué pasa si falta un actual?

Retiré temporalmente Formato.android.kt y ejecuté :shared:compileAndroidMain --offline. El compilador detuvo la tarea con el mensaje literal: Expected formatearSoles has no actual declaration in module <commonMain> for JVM. La línea completa comenzó e: file:///C:/Users/LENOVO/Desktop/DAM/PHARMOBILE%20ULTIMO/PharmaMobile_Valencia/shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobile/platform/Formato.kt:4:1. Gradle informó BUILD FAILED in 5s y la tarea :shared:compileAndroidMain FAILED. Esto ocurre al compilar, antes de ejecutar la app: Kotlin exige que la promesa de commonMain tenga una implementación para el target. Capturé el mismo error en Build Output de Android Studio y restauré el archivo original. La captura auténtica está en Evidencias/04-error-expect-sin-actual-android-studio.jpg.

5. ¿Qué no debe bajar a código de plataforma?

La validación del formulario de producto no debería separarse entre Android e iOS. Por ejemplo, ProductoViewModel.kt y el estado ProductoUi.kt trabajan con los mismos datos de nombre, precio, stock y categoría que envía PharmaSoft. La regla de que un nombre inválido no puede guardarse y la lógica de filtrado de productos activos o de bajo stock deben comportarse igual en los dos dispositivos. Llevarlas a androidMain e iosMain duplicaría reglas y permitiría que un mismo producto se acepte en uno y se rechace en otro. Lo mismo ocurre con la construcción del mensaje compartido en TextoParaCompartir.kt: el contenido debe ser único; solo la acción de abrir el menú necesita código nativo. El criterio es identificar si cambia una regla del negocio o la API necesaria para interactuar con el sistema operativo. Solo lo segundo baja al source set específico.

Tercera capacidad y conclusión

InfoDispositivo.kt añade expect class InfoDispositivo() con sistema y version. Android obtiene Build.VERSION.RELEASE; iOS usa UIDevice.currentDevice. El botón «Acerca de este dispositivo» de ProductoScreen.kt muestra esos datos en un diálogo. Android pasó compilación y pruebas; la captura real del emulador mostró «Android» y «17». El código iOS no se verificó en Xcode. Así, la mayor parte de la interfaz sigue compartida y la dependencia directa de cada sistema queda confinada a androidMain e iosMain.

Producto 3 · Información nativa del dispositivo

El contrato expect class InfoDispositivo() vive en commonMain. Su actual Android obtiene la versión mediante Build.VERSION.RELEASE y la implementación iOS usa UIDevice.currentDevice. ProductoScreen añade el botón «Acerca de este dispositivo» y muestra sistema y versión en un cuadro. La ejecución real en el emulador mostró «Android» y «17». La implementación iOS no se verificó en Xcode.

Comprobaciones: :shared:testAndroidHostTest y :androidApp:assembleDebug terminaron con BUILD SUCCESSFUL. La búsqueda de imports android. y platform. en commonMain no devolvió coincidencias. En la captura de esta pantalla, el backend estaba apagado: el error de conexión visible detrás del diálogo no afecta a la información nativa del dispositivo.

Producto 4 · Evidencias Android

Figura 1. Listado Android: precio de producto formateado en soles.



Figura 2. Selector nativo Android con el texto del producto.

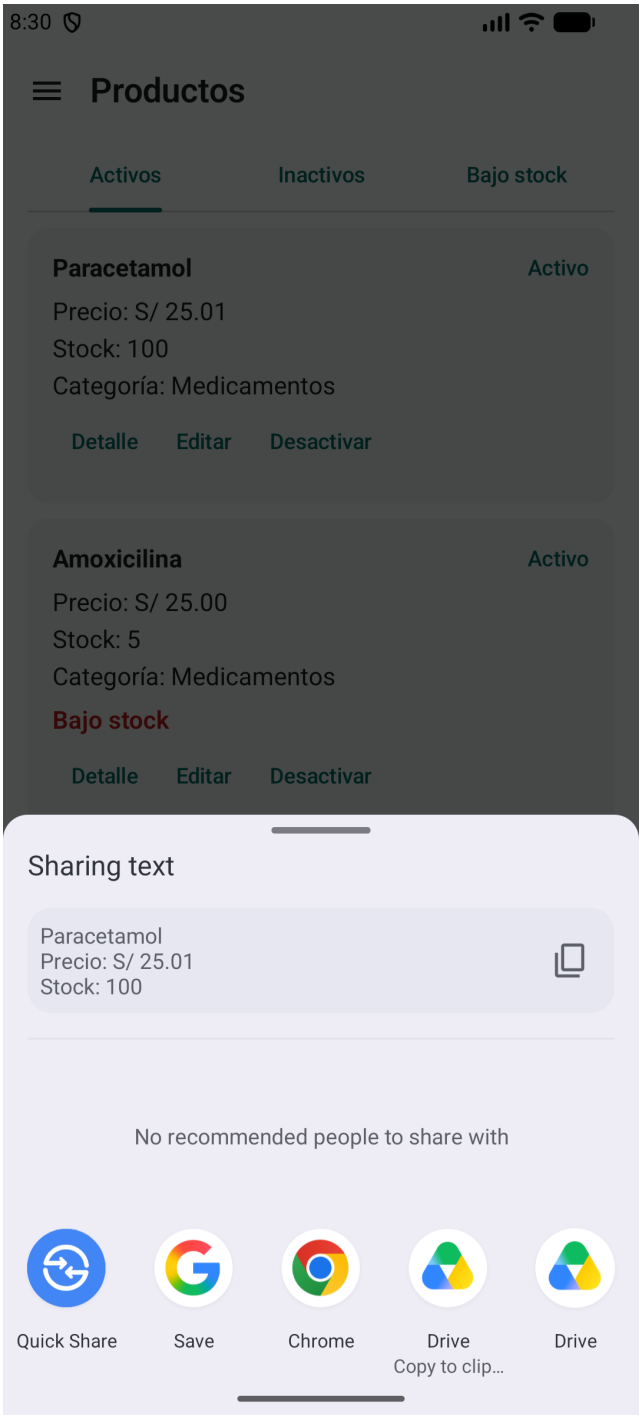


Figura 3. Nueva capacidad: sistema Android y versión 17 en la aplicación.

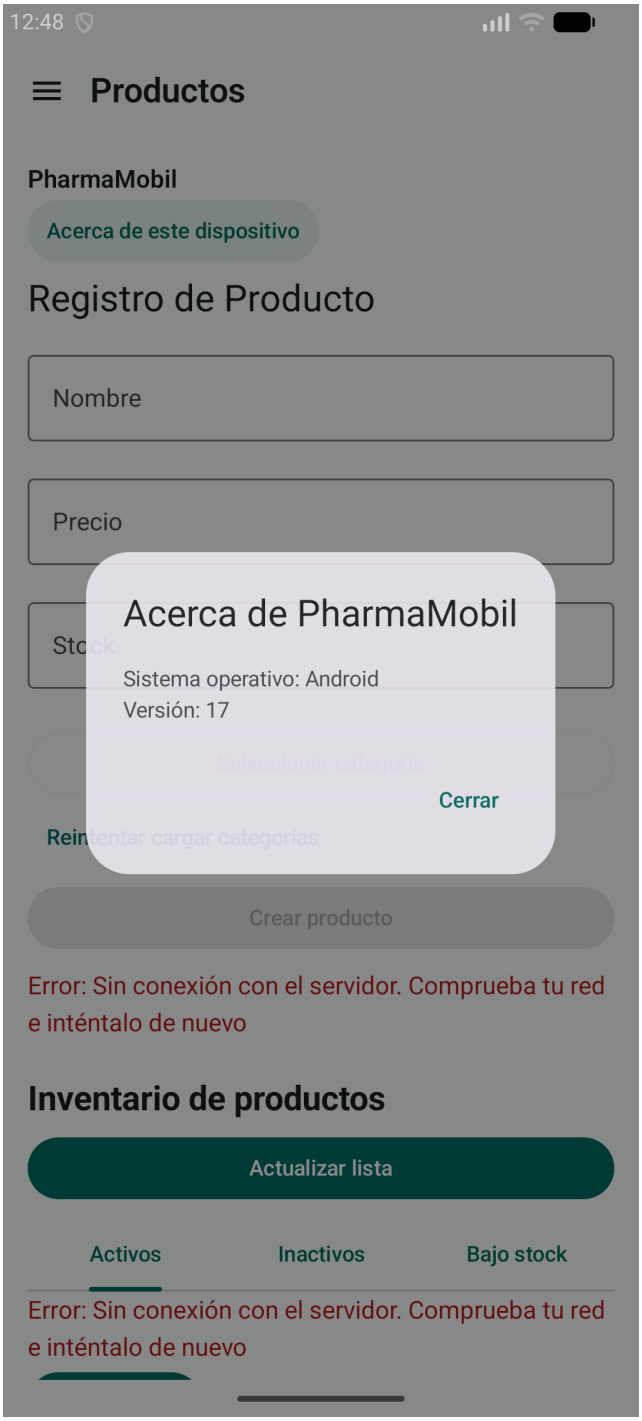
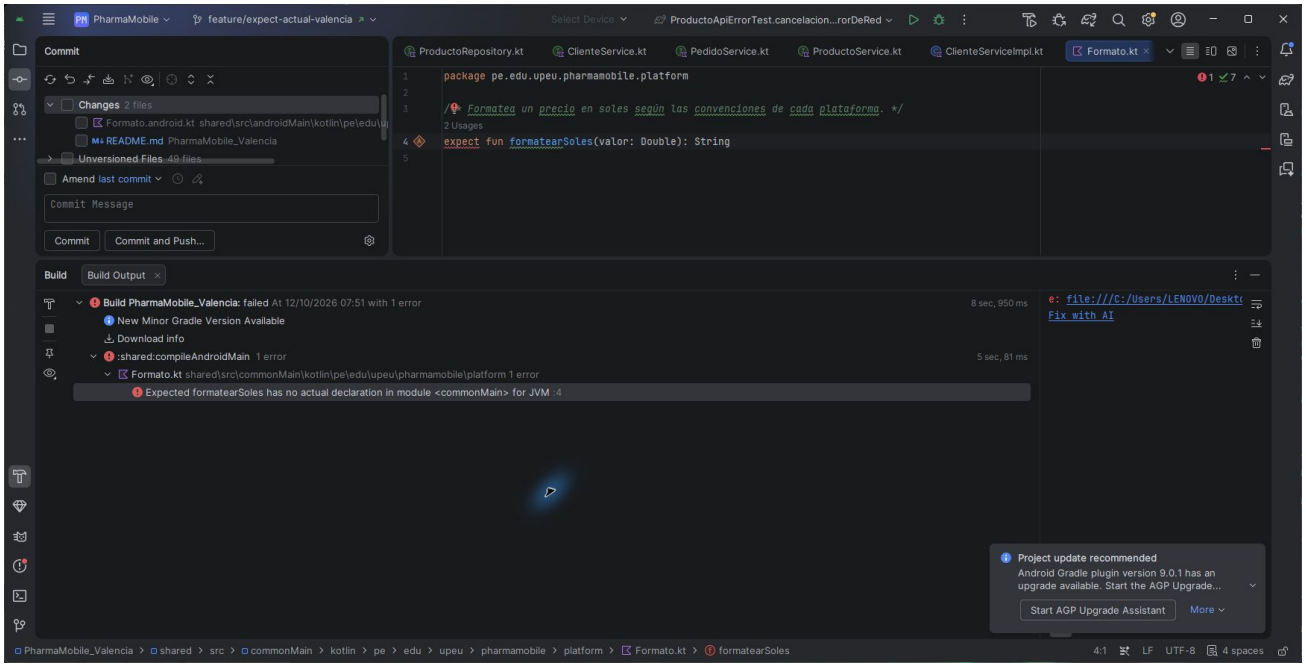


Figura 4. Build Output real: error al faltar el actual de formatearSoles para JVM.



Repositorio y entrega

Rama: https://github.com/Akatosh4019/PharmaMobile_Valencia/tree/feature/expect-actual-valencia. La carpeta Unidad 2/Sesion 3 contiene este PDF, los archivos fuente del inventario e informe, y las cuatro capturas. No se adjuntan capturas iOS ni se afirma equivalencia funcional verificada en esa plataforma.